

Lekcja 14: Zaawansowane SQL i wprowadzenie do ORM

#lekcja #python #sql #bazy_danych #orm

Witaj w kolejnej lekcji! Dzisiaj zagłębimy się w bardziej zaawansowane techniki języka SQL, które pozwolą Ci na pisanie skomplikowanych i wydajnych zapytań. Nauczymy się, jak używać podzapytań, funkcji agregujących oraz jak łączyć dane z wielu tabel. Na koniec wprowadzimy Cię w świat ORM, czyli technologii, która łączy bazy danych z programowaniem obiektowym w Pythonie.

1. Podzapytania (Subqueries)

Czasami, aby odfiltrować dane, potrzebujemy wartości, którą musimy najpierw obliczyć w osobnym zapytaniu. Zamiast wykonywać dwa osobne zapytania, możemy zagnieździć jedno w drugim.

Definition

Podzapytanie (ang. subquery), nazywane też zapytaniem wewnętrznym lub zagnieżdżonym, to zapytanie SQL umieszczone wewnątrz innego, głównego zapytania. Najczęściej znajduje się w klauzuli WHERE. Wynik podzapytania jest wykorzystywany przez zapytanie główne jako warunek do dalszego filtrowania danych.

Wyobraź sobie, że chcesz znaleźć wszystkie produkty, których cena jest wyższa niż średnia cena wszystkich produktów w sklepie. Aby to zrobić, musisz najpierw obliczyć średnią cenę, a dopiero potem użyć jej do filtrowania.

Przykład:

```
import sqlite3

# Utworzenie połączenia z bazą danych w pamięci
conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Utworzenie tabeli i dodanie danych
cursor.execute('''
CREATE TABLE Produkty (
    id INTEGER PRIMARY KEY,
    nazwa TEXT NOT NULL,
    cena REAL NOT NULL
```

```

)
'''
produkty = [
    ('Laptop', 4500.00),
    ('Myszka', 150.00),
    ('Klawiatura', 250.00),
    ('Monitor', 1200.00),
    ('Słuchawki', 350.00)
]
cursor.executemany('INSERT INTO Produkty (nazwa, cena) VALUES (?, ?)',
produkty)
conn.commit()

# Zapytanie z podzapytaniem
# Zapytanie główne: SELECT * FROM Produkty WHERE cena > (...)
# Podzapytanie: (SELECT AVG(cena) FROM Produkty)
query = '''
SELECT nazwa, cena
FROM Produkty
WHERE cena > (SELECT AVG(cena) FROM Produkty)
'''

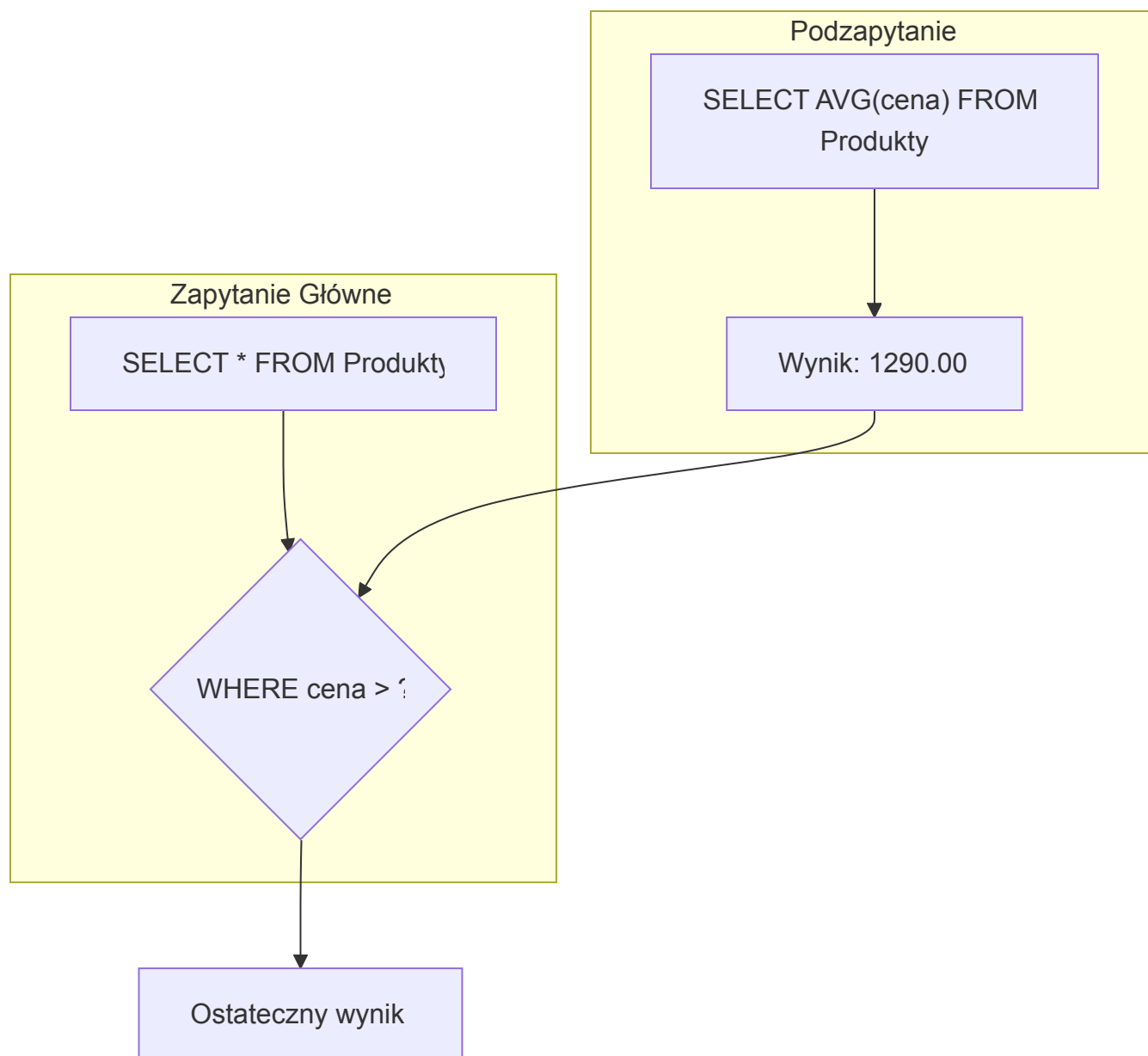
cursor.execute(query)
wyniki = cursor.fetchall()

print("Produkty droższe niż średnia:")
for row in wyniki:
    print(f"- {row[0]}: {row[1]:.2f} zł") # Laptop, Monitor

conn.close()

```

Schemat działania:



2. Funkcje agregujące w SQL

Funkcje agregujące wykonują obliczenia na zbiorze wierszy i zwracają pojedynczą wartość. Są niezwykle przydatne do tworzenia podsumowań i raportów.

Definition

Funkcje agregujące (ang. aggregate functions) to specjalne funkcje w SQL, które operują na grupie wierszy i zwracają jedną, zagregowaną wartość. Jeśli zapytanie nie zwróci żadnych wierszy, wynikiem funkcji będzie NULL (z wyjątkiem COUNT).

Najważniejsze funkcje agregujące:

- `COUNT()` – zlicza liczbę wierszy.

- `AVG()` – oblicza średnią wartość w kolumnie.
- `SUM()` – oblicza sumę wartości w kolumnie.
- `MIN()` – znajduje minimalną wartość w kolumnie.
- `MAX()` – znajduje maksymalną wartość w kolumnie.

Przykład:

```
import sqlite3

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

cursor.execute('''
CREATE TABLE Produkty (
    id INTEGER PRIMARY KEY,
    nazwa TEXT NOT NULL,
    cena REAL NOT NULL,
    ilosc_magazyn INT NOT NULL
)
''')

produkty = [
    ('Laptop', 4500.00, 10),
    ('Myszka', 150.00, 150),
    ('Klawiatura', 250.00, 70),
    ('Monitor', 1200.00, 30),
    ('Słuchawki', 350.00, 90)
]

cursor.executemany('INSERT INTO Produkty (nazwa, cena, ilosc_magazyn) VALUES (?, ?, ?, ?)', produkty)
conn.commit()

# Zapytanie używające funkcji agregujących
query = '''
SELECT
    COUNT(id) as liczba_produktoiw,
    AVG(cena) as srednia_cena,
    SUM(ilosc_magazyn) as laczna_ilosc_w_magazynie,
    MIN(cena) as najnizsza_cena,
    MAX(cena) as najwyzsza_cena
FROM Produkty
'''

cursor.execute(query)
wynik = cursor.fetchone() # Zwraca jeden wiersz z wynikami
```

```
print(f"Liczba wszystkich produktów: {wynik[0]}")
print(f"Średnia cena produktu: {wynik[1]:.2f} zł")
print(f"Łączna ilość sztuk w magazynie: {wynik[2]}")
print(f"Najniższa cena: {wynik[3]:.2f} zł")
print(f"Najwyższa cena: {wynik[4]:.2f} zł")

conn.close()
```

Info

Funkcje agregujące są często używane z klauzulą GROUP BY, która grupuje wiersze o tych samych wartościach w określonych kolumnach w celu wykonania agregacji na każdej z tych grup.

3. Łączenie tabel – JOIN

Rzadko kiedy wszystkie potrzebne informacje znajdują się w jednej tabeli. Operator JOIN pozwala na łączenie wierszy z dwóch lub więcej tabel na podstawie powiązanej między nimi kolumny.

Definition

JOIN to klauzula SQL, która służy do łączenia wierszy z dwóch lub więcej tabel na podstawie wspólnej kolumny (klucza). Pozwala na tworzenie zapytań, które pobierają dane z wielu powiązanych ze sobą tabel jednocześnie.

Najczęściej używanym typem jest INNER JOIN, który zwraca tylko te wiersze, dla których istnieje dopasowanie w obu tabelach.

Przykład:

Założmy, że mamy tabelę Klienci i tabelę Zamowienia. Chcemy wyświetlić listę zamówień wraz z imionami klientów, którzy je złożyli.

```
import sqlite3

conn = sqlite3.connect(':memory:')
cursor = conn.cursor()

# Tabela klientów
cursor.execute('''
CREATE TABLE Klienci (
```

```

        id_klienta INTEGER PRIMARY KEY,
        imie TEXT NOT NULL
    )'''
cursor.execute("INSERT INTO Klienci (imie) VALUES ('Anna'), ('Piotr'),
('Katarzyna')")

# Tabela zamówień
cursor.execute('''
CREATE TABLE Zamowienia (
    id_zamowienia INTEGER PRIMARY KEY,
    id_klienta INTEGER,
    produkt TEXT NOT NULL,
    FOREIGN KEY (id_klienta) REFERENCES Klienci(id_klienta)
)''')
cursor.execute("INSERT INTO Zamowienia (id_klienta, produkt) VALUES (1,
'Laptop'), (2, 'Myszka'), (1, 'Monitor')")
conn.commit()

# Zapytanie z JOIN
query = '''
SELECT
    Klienci.imie,
    Zamowienia.produkt
FROM Zamowienia
INNER JOIN Klienci ON Zamowienia.id_klienta = Klienci.id_klienta
'''

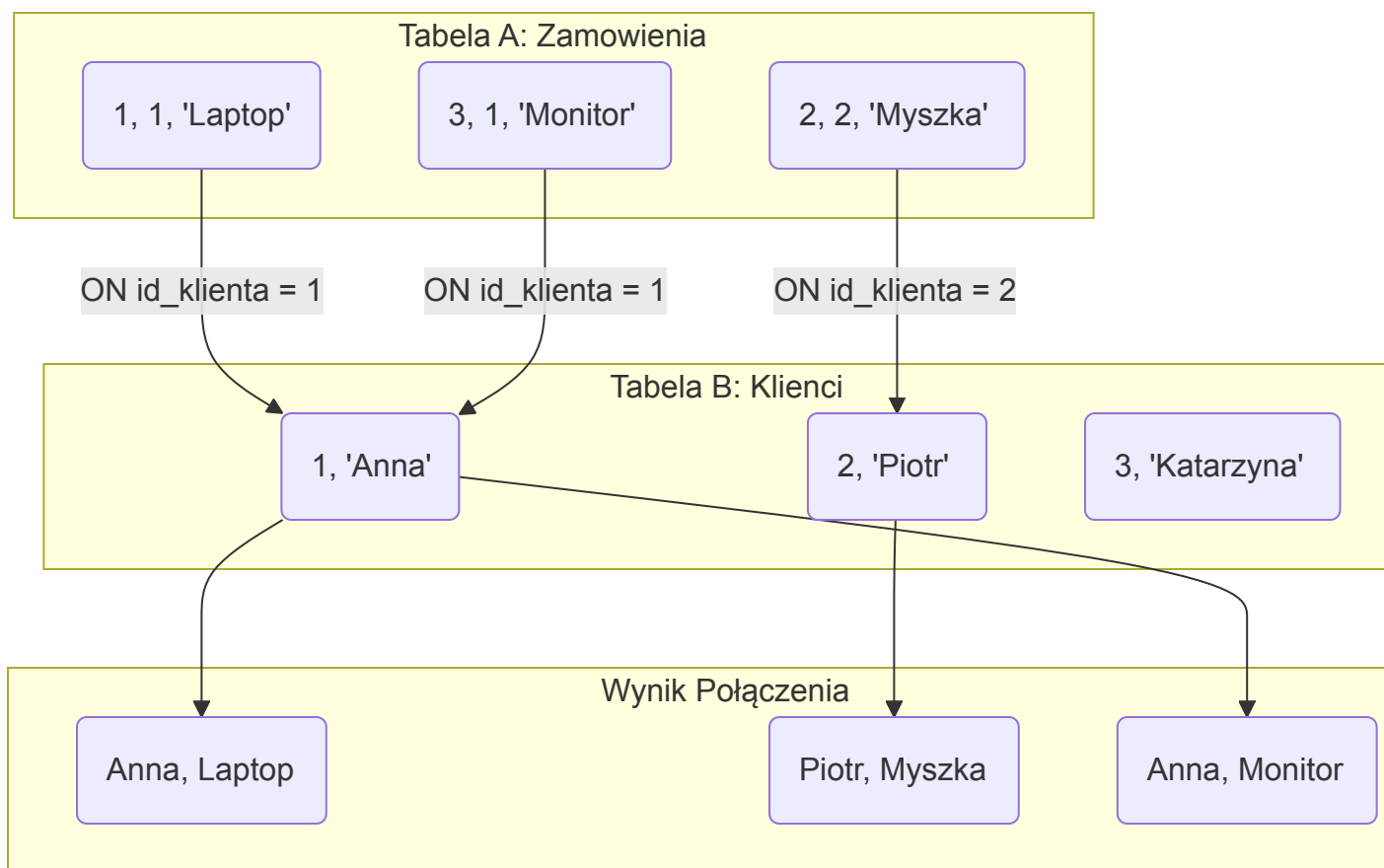
cursor.execute(query)
wyniki = cursor.fetchall()

print("Zamówienia klientów:")
for row in wyniki:
    print(f"- Klient: {row[0]}, Produkt: {row[1]}")

conn.close()

```

Schemat działania INNER JOIN :



4. ORM (Object-Relational Mapping)

Na koniec porozmawiamy o technologii, która stanowi pomost między światem relacyjnych baz danych (SQL) a światem programowania obiektowego (np. w Pythonie).

Definition

ORM (Object-Relational Mapping) to technika programistyczna, która pozwala na interakcję z bazą danych przy użyciu obiektów i metod zamiast pisania "surowych" zapytań SQL. ORM "tłumaczy" operacje na obiektach (np. tworzenie instancji klasy, zmiana atrybutu) na odpowiednie komendy SQL (INSERT, UPDATE itp.).

Zamiast pisać: `INSERT INTO Uzytkownicy (imie, email) VALUES ('Jan', 'jan@example.com')`, w ORM moglibyśmy napisać coś w stylu:

```
# To jest pseudokod, nie zadziała bez biblioteki ORM!
nowy_uzytkownik = Uzytkownik(imie='Jan', email='jan@example.com')
session.add(nowy_uzytkownik)
session.commit()
```

Popularne biblioteki ORM w Pythonie to **SQLAlchemy** i **Peewee**.

Zalety ORM:

- **Wygoda i szybkość:** Pisanie kodu w Pythonie jest często szybsze i bardziej intuicyjne niż tworzenie skomplikowanych zapytań SQL.
- **Abstrakcja bazy danych:** ORM potrafi generować kod dla różnych dialektów SQL (SQLite, PostgreSQL, MySQL). Zmiana bazy danych staje się łatwiejsza.
- **Bezpieczeństwo:** ORM pomaga chronić przed atakami typu SQL Injection, ponieważ parametryzuje zapytania.
- **Integracja z kodem:** Dane z bazy są od razu dostępne jako obiekty Pythona, co ułatwia pracę.

Wady ORM:

- **Utrata zrozumienia:** Programiści mogą przestać rozumieć, co faktycznie dzieje się "pod spodem". Nie wiedzą, jakie zapytania SQL są generowane, co może prowadzić do problemów z wydajnością.
- **Wydajność:** Wygenerowane przez ORM zapytania mogą być wolniejsze niż te napisane ręcznie przez doświadczonego programistę bazodanowego.
- **Ograniczenia:** Przy bardzo skomplikowanych, niszowych zapytaniach ORM może sobie nie poradzić i konieczne będzie napisanie "surowego" SQL.

Zadania do samodzielnej pracy

Aby wykonać poniższe zadania, najpierw utwórz i wypełnij swoją bazę danych za pomocą poniższego kodu. Użyj go jako punktu wyjścia dla każdego zadania.

```
# ----- KOD STARTOWY DO ZADAŃ -----
import sqlite3

def przygotuj_baze():
    """Tworzy i wypełnia bazę danych na potrzeby zadań."""
    conn = sqlite3.connect('sklep.db') # Tworzy plik sklep.db
    cursor = conn.cursor()

    # Usunięcie tabel, jeśli istnieją, dla czystego startu
    cursor.execute("DROP TABLE IF EXISTS Zamowienia_Produkty")
    cursor.execute("DROP TABLE IF EXISTS Zamowienia")
    cursor.execute("DROP TABLE IF EXISTS Produkty")
    cursor.execute("DROP TABLE IF EXISTS Kategorie")
    cursor.execute("DROP TABLE IF EXISTS Klienci")
```



```

# Tworzenie tabel
cursor.execute('''
CREATE TABLE Kategorie (
    id_kategorii INTEGER PRIMARY KEY,
    nazwa_kategorii TEXT UNIQUE NOT NULL
)''')
cursor.execute('''
CREATE TABLE Produkty (
    id_produktu INTEGER PRIMARY KEY,
    nazwa_produktu TEXT NOT NULL,
    cena REAL NOT NULL,
    id_kategorii INTEGER,
    FOREIGN KEY (id_kategorii) REFERENCES Kategorie(id_kategorii)
)''')
cursor.execute('''
CREATE TABLE Klienci (
    id_klienta INTEGER PRIMARY KEY,
    imie TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
)''')
cursor.execute('''
CREATE TABLE Zamowienia (
    id_zamowienia INTEGER PRIMARY KEY,
    id_klienta INTEGER,
    data_zamowienia DATE,
    FOREIGN KEY (id_klienta) REFERENCES Klienci(id_klienta)
)''')
cursor.execute('''
CREATE TABLE Zamowienia_Produkty (
    id_zamowienia INTEGER,
    id_produktu INTEGER,
    ilosc INTEGER NOT NULL,
    PRIMARY KEY (id_zamowienia, id_produktu),
    FOREIGN KEY (id_zamowienia) REFERENCES Zamowienia(id_zamowienia),
    FOREIGN KEY (id_produktu) REFERENCES Produkty(id_produktu)
)''')

# Wstawianie danych
kategorie = [('Elektronika',), ('Książki',), ('Dom i ogród',)]
klienci = [('Anna Nowak', 'anna.n@example.com'), ('Jan Kowalski',
'jan.k@example.com'), ('Zofia Wiśniewska', 'zofia.w@example.com')]
produkty = [
    ('Laptop Pro', 5200.00, 1), ('Smartfon X', 2500.00, 1),
    ('Python dla każdego', 89.99, 2), ('Wzorce projektowe', 120.50, 2),
    ('Kosiarka elektryczna', 750.00, 3), ('Zestaw narzędzi', 300.00, 3),

```

```

        ('Słuchawki bezprzewodowe', 450.00, 1)
    ]
    zamowienia = [(1, '2023-10-01'), (2, '2023-10-02'), (1, '2023-10-05')]
    zamowienia_produkty = [(1, 1, 1), (1, 7, 1), (2, 3, 2), (3, 5, 1)]

    cursor.executemany("INSERT INTO Kategorie (nazwa_kategorii) VALUES (?)",
kategorie)
    cursor.executemany("INSERT INTO Klienci (imie, email) VALUES (?,?)",
klienci)
    cursor.executemany("INSERT INTO Produkty (nazwa_produkty, cena,
id_kategorii) VALUES (?,?,?)", produkty)
    cursor.executemany("INSERT INTO Zamowienia (id_klienta, data_zamowienia)
VALUES (?,?)", zamowienia)
    cursor.executemany("INSERT INTO Zamowienia_Produkty (id_zamowienia,
id_produkty, ilosc) VALUES (?,?,?)", zamowienia_produkty)

    conn.commit()
    conn.close()
    print("Baza 'sklep.db' została przygotowana.")

# Wywołaj funkcję, aby stworzyć bazę przed rozpoczęciem pracy
przygotuj_baze()

```

Zadania proste

1. Zadanie 1 – Liczba produktów

Napisz skrypt, który połączy się z bazą sklep.db i policzy, ile jest wszystkich produktów w tabeli Produkty. Użyj funkcji COUNT().

(proste)

2. Zadanie 2 – Najdroższy produkt

Napisz skrypt, który znajdzie nazwę i cenę najdroższego produktu w sklepie. Użyj funkcji MAX().

(proste)

3. Zadanie 3 – Suma wartości

Oblicz i wyświetl łączną wartość wszystkich produktów z kategorii "Elektronika". Użyj funkcji SUM() oraz klauzuli WHERE z JOIN.

(proste)

4. Zadanie 4 – Średnia cena książki

Napisz zapytanie, które obliczy średnią cenę produktów w kategorii "Książki". Użyj AVG().

(proste)

5. 🖋️ Zadanie 5 – Lista klientów

Napisz skrypt, który wyświetli imiona i adresy e-mail wszystkich klientów z tabeli Klienci.

(proste)

Zadania "challenge"

6. 🧠 Zadanie 6 – Produkty droższe od średniej

Napisz skrypt, który wyświetli nazwy i ceny wszystkich produktów, których cena jest wyższa niż średnia cena wszystkich produktów w sklepie. Wykorzystaj podzapytanie.

(challenge)

7. 🧠 Zadanie 7 – Zamówienia Anny Nowak

Napisz skrypt, który wyświetli nazwy wszystkich produktów zamówionych przez klienta o imieniu 'Anna Nowak'. Będziesz potrzebować połączyć dane z czterech tabel: Klienci, Zamowienia, Zamowienia_Produkty i Produkty.

(challenge)

8. 🧠 Zadanie 8 – Kategorie z liczbą produktów

Napisz zapytanie, które wyświetli nazwę każdej kategorii oraz liczbę produktów należących do tej kategorii. Użyj JOIN, COUNT() oraz GROUP BY.

(challenge)

9. 🧠 Zadanie 9 – Funkcja do wyszukiwania produktów

Napisz w Pythonie funkcję `znajdz_produkty_w_kategorii(nazwa_kategorii)`, która przyjmuje jako argument nazwę kategorii i zwraca listę krotek (`nazwa_produkту, cena`) dla wszystkich produktów w tej kategorii.

(challenge)

10. 🧠 Zadanie 10 – Prosta symulacja ORM

Stwórz klasę `Produkt` w Pythonie z atrybutami `id_produkту`, `nazwa_produkту` i `cena`. Następnie napisz funkcję `pobierz_wszystkie_produkty()`, która połączy się z bazą danych, pobierze wszystkie produkty i zwróci listę obiektów klasy `Produkt`. To ćwiczenie pokaże Ci, jak ORM automatyzuje mapowanie wierszy na obiekty.

(challenge)